

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Register Number	192312188
Name	X. Joshua

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks: 50

Code of Conduct:

I X. Joshua-(192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks: Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario

An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks: Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario

- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
Total					50

ANSWERS

Answer 1: AI Drone Delivery in Flood-Affected Region

i. Greedy Best-First Search (GBFS) Behaviour

Greedy Best-First Search selects, at every step, the node that appears closest to the goal according to the heuristic $h(n)$ — here, the straight-line distance to the delivery point — completely ignoring the actual path cost $g(n)$ already incurred. In the flood scenario, the drone would repeatedly choose the neighbouring waypoint with the smallest straight-line distance to the destination, moving very quickly through the search space and producing a route with almost no computational delay, which suits the urgency of emergency delivery.

Strengths: GBFS is fast and memory-efficient because it expands very few nodes when the heuristic is reasonably informative; this makes it attractive when a decision is needed within seconds. Weaknesses: because it never reconsiders $g(n)$, GBFS can be misled by a route that looks close in straight-line terms but is actually blocked, waterlogged, or has a very high real terrain cost. Under uncertainty (sudden blocked roads, changing weather), GBFS has no mechanism to recover the true cost of a path once it has committed to a heuristically attractive but practically expensive or dangerous direction — it is neither complete nor optimal, and can loop or get trapped near obstacles such as flooded zones.

ii. A* Search Behaviour

A* evaluates every node using $f(n) = g(n) + h(n)$, combining the accumulated real travel cost $g(n)$ with the estimated remaining cost $h(n)$. For the drone, $g(n)$ would capture the actual, weather- and terrain-adjusted cost of the path travelled so far, while $h(n)$ remains the straight-line distance estimate. By expanding the node with the lowest $f(n)$, A* balances 'how far I have already paid' against 'how far I still expect to go', so it will not blindly chase a heuristically close node if reaching it has been very costly.

This balance allows A* to systematically explore alternate routes when the low-cost-looking path turns out to be expensive, and — provided the heuristic is admissible (never overestimates the true remaining cost) and consistent — A* is guaranteed to find the optimal (minimum-cost) route to the delivery point, which is critical when cost is tied to real flood risk and delay.

iii. Impact of Heuristic Accuracy

For GBFS, heuristic accuracy is everything: since path cost is ignored, an inaccurate or misleading heuristic (e.g., straight-line distance across a flooded lake that is impassable by road) directly causes poor, sometimes disastrous, route choices. A highly accurate heuristic makes GBFS behave almost optimally by luck; a poor heuristic makes it behave arbitrarily badly.

For A*, heuristic accuracy affects efficiency rather than correctness. As long as the heuristic remains admissible, A* still guarantees an optimal solution even if the heuristic is only loosely informative — it will simply expand more nodes and take longer. A tighter, more accurate admissible heuristic reduces the number of nodes expanded and speeds up convergence, which matters for the drone's real-time constraint, while an overestimating (inadmissible) heuristic can break A*'s optimality guarantee.

iv. Effect of Dynamic Changes (Blocked Paths)

Both algorithms are traditionally designed for static graphs, so a sudden blockage invalidates part of the previously computed path. GBFS, having committed greedily, may need to restart its search from scratch with little reusable information, and can behave erratically if it keeps re-selecting the same heuristically attractive but now-blocked direction. A* is more robust because its search maintains cost information (g -values) for all visited nodes, so when a path is blocked, the frontier can be updated and re-expanded using open/closed lists, and variants such as D* Lite or Anytime Repairing A* are specifically built to replan efficiently as the environment changes. In general, dynamic blockages increase node expansions, force replanning, and can transiently break optimality unless an incremental/real-time search variant is used.

v. Recommendation

A* is the more suitable algorithm for this scenario. While GBFS offers speed, its disregard for actual path cost is dangerous in a life-critical flood-relief context where a 'short-looking' route may be blocked or hazardous. A* preserves near-real-time performance (especially with a well-tuned admissible heuristic) while still guaranteeing an optimal, safer route, and it degrades gracefully into a replanning framework (e.g., D* Lite) to handle the partial, dynamic real-time updates about blocked paths. The small extra computational cost of A* is a reasonable trade-off for the gain in optimality and safety.

Answer 2: Smart City Traffic Signal Optimization

Problem Formulation and Why Traditional Search is Inefficient

This is naturally formulated as a local-search / optimization problem rather than a path-finding problem. A candidate state is a complete assignment of signal timings (green/red durations and offsets) to every intersection; the state space is the set of all such assignments. The objective function to be minimized is a weighted combination of total waiting time, fuel consumption, and congestion (equivalently, maximized as the negative of this cost). There is no single fixed 'goal state' or clear path from a start to a goal as in classical search — instead, the system searches for the best point in a huge, continuous/near-continuous configuration space that also keeps changing over time.

Traditional systematic search (BFS, DFS, A*) is inefficient here because the branching factor (possible timing combinations across hundreds of intersections) is astronomically large, there is no natural goal test, path cost to a 'goal' is undefined, and the environment is non-stationary (traffic patterns change continuously), so a solution computed offline can become stale before it is even applied. Optimization/local-search methods that only track the current best configuration and use path-independent heuristics are far more scalable.

i. Hill Climbing and Its Limitations

Hill Climbing starts from a current signal-timing configuration and repeatedly moves to a neighbouring configuration (e.g., adjusting one intersection's green time slightly) only if it improves the objective (reduces waiting time/fuel/congestion). It is simple, memory-light, and can adapt reasonably quickly to small traffic changes. However, it is a purely local method: it has no mechanism to escape a configuration that is only locally good, cannot look ahead across the network-wide interactions between intersections, and provides no guarantee of finding a globally good timing plan.

ii. Local Maxima, Plateaus, and Ridges

A local maximum (in maximization terms) corresponds to a timing configuration where every small adjustment to any single intersection makes things worse, even though a very different configuration elsewhere in the space would perform much better — for example, a plan that is efficient for a few major arterial roads but subtly congests several minor streets, where no single-signal tweak improves things unless several signals change together.

A plateau occurs when many neighbouring configurations produce almost identical waiting times — e.g., adjusting a signal by one or two seconds has no measurable real-world effect — so Hill Climbing has no gradient to follow and wanders or stalls. A ridge occurs when the true improving direction requires simultaneous, coordinated changes across several intersections (e.g., 'green-wave' synchronization along a corridor), but Hill Climbing only ever changes one dimension at a time and therefore cannot move directly along that diagonal improving direction, making progress slow and zig-zagging.

iii. Simulated Annealing and Genetic Algorithms as Solutions

Simulated Annealing (SA) allows occasional moves to worse configurations, with acceptance probability decreasing as a 'temperature' parameter cools over time. This lets the system escape local maxima and plateaus early in the search while gradually settling into a strong solution, which suits gradually stabilizing signal plans as more traffic data arrives.

Genetic Algorithms (GA) maintain a population of many candidate timing plans simultaneously, combine good partial plans through crossover, and introduce diversity through mutation. Because GA explores multiple regions of the search space in parallel and can recombine good coordinated sub-patterns (such as a well-synchronized corridor) from different candidates, it is naturally suited to overcoming ridges and local maxima that trap single-point methods like Hill Climbing.

iv. Recommendation

A Genetic Algorithm (optionally hybridized with local search, i.e., a 'memetic algorithm') is the best overall approach for city-scale scalability, since it explores many intersections' configurations in parallel and scales naturally by partitioning the population across sub-regions of the city. For faster, more localized real-time re-tuning between full GA re-optimization cycles, Simulated Annealing can be used as a lightweight adaptive layer. This combination balances the adaptability required by constantly changing traffic patterns with the scalability required by hundreds of intersections, something plain Hill Climbing cannot offer.

Answer 3: Autonomous Mars Rover

i. Why an Online Search Agent is Required

Offline search assumes the agent has complete, correct knowledge of the environment in advance and can compute an entire solution path before acting. The rover, however, has no complete map of the Martian terrain and cannot predict hazards such as hidden craters or loose rocks until it actually senses them. This forces the rover to interleave computation with action: it must act, observe the actual outcome, update its model of the world, and then decide the next action — which is precisely the definition of an online search agent. An offline planner would produce a plan that is likely to become invalid the moment an unmapped hazard is encountered.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability means the rover's sensors reveal only the immediate surroundings, not the full state of the terrain, so it must make decisions under genuine uncertainty about what lies beyond its sensing range — a path that looks safe nearby could lead to an impassable region further ahead. Dynamic aspects such as dust storms, shifting sand, or rockslides mean that even previously explored terrain cannot always be trusted to remain unchanged. Together, these force the rover to continually balance exploration (gaining new information) against exploitation (making progress toward the sample-collection goal), and to treat every plan as provisional and revisable.

iii. Building and Updating the Internal Model

The rover incrementally builds an internal map (e.g., an occupancy grid or topological graph) from its sensor readings — cameras, LIDAR, and proximity sensors. Each time it moves and senses, it marks confirmed-safe cells, marks hazards as blocked, and leaves unexplored areas as unknown. This model is continuously revised: if a previously assumed-passable cell turns out to be blocked, the rover updates that node's status and, if necessary, backtracks to a previously visited state to try an alternative route (similar to the way an online-DFS or LRTA* agent maintains and revises cost estimates for visited states).

iv. Online Search vs Uninformed and Informed Search

Uninformed search (BFS/DFS) explores blindly without using any domain knowledge of distance or direction to the goal, and both assume the full state space graph is already known — unsuitable when the map is not available in advance. Informed offline search (like A*) uses a heuristic to search efficiently but still assumes the whole graph and costs are known ahead of time so it can compute a full path before execution. Online search (e.g., Online DFS, LRTA*) differs from both because it must act first and learn as it goes, using only locally available information and revising its knowledge after every action, at the cost of sometimes-suboptimal or repeated exploration compared to a hypothetical agent with full prior knowledge.

v. Justification of the Most Suitable Approach

An online, informed search approach such as Learning Real-Time A* (LRTA*) is most suitable: it combines a heuristic (informed guidance toward the sample site, improving efficiency over blind exploration) with the online property of acting under partial observability and updating its cost estimates as hazards are discovered (handling uncertainty and dynamism safely). This gives the rover a good balance of adaptability (replanning when the model changes), safety (avoiding acting purely reactively without any goal-directed heuristic), and reasonable efficiency compared to purely uninformed online exploration.

Answer 4: University Exam Timetabling as a CSP

i. Variables, Domains, and Constraints

Variables: Each variable represents an entity that must be scheduled — the most natural formulation defines one variable per course/exam, e.g., Exam1, Exam2, ..., ExamN, each of which must be assigned a (time-slot, hall) pair. Additional variables can represent faculty invigilator assignments if faculty scheduling is modelled explicitly.

Domains: The domain of each Exam variable is the set of all valid (time-slot, hall) combinations available during the examination period, e.g., {(Day1-Slot1, Hall A), (Day1-Slot1, Hall B), (Day1-Slot2, Hall A), ...}. The domain size shrinks as constraints are applied (for example, a hall too small for the enrolled class size is removed from that exam's domain in advance).

Constraints: Unary constraints restrict a single exam's domain directly, e.g., a hall's capacity must be at least the number of students enrolled, and faculty-availability restricts which invigilator-linked slots are valid. Binary/ n-ary constraints link multiple exams: no two exams sharing a common student may be assigned the same time-slot (student non-overlap); two exams cannot be assigned the same hall at the same time-slot (hall exclusivity); certain subject pairs have a precedence constraint

(subject A's exam slot must occur strictly before subject B's); and special-accommodation students require specific slots/halls (e.g., extended time or ground-floor access), which further restricts the domains of exams they are enrolled in.

ii. Backtracking Search in This Scenario

Backtracking search assigns exams one at a time (e.g., in some variable order), checking after each assignment whether all constraints involving already-assigned exams are still satisfied. If assigning Exam_k to a (slot, hall) pair violates a constraint (say, it creates a student overlap with an already-scheduled exam), that value is rejected and the next candidate value in the domain is tried; if no value works, the search backtracks to the previous exam and tries a different assignment for it. This depth-first, constraint-checking process continues until either a complete, consistent timetable is found or the search exhausts all possibilities (indicating the constraints as given are infeasible).

iii. Constraint Propagation (e.g., Forward Checking)

Forward checking looks ahead: whenever an exam is assigned a (slot, hall), it immediately removes now-inconsistent values from the domains of all as-yet-unassigned exams that are directly constrained with it (for example, removing that same slot from the domain of every other exam sharing a student with the one just scheduled). This prunes the search tree early, detecting dead-ends far sooner than plain backtracking, which would otherwise only discover the conflict once it tries to assign the conflicting exam directly. More powerful propagation, such as arc-consistency (AC-3), goes further by repeatedly enforcing consistency between every pair of constrained exams until no domain can be reduced further, substantially cutting down the search space before backtracking even begins.

iv. Real-World Challenges and Best Strategy for Scalability

As the number of courses, students, and halls grows, the constraint graph becomes extremely dense (many students taking overlapping combinations of courses), which increases both the size of the search tree and the cost of constraint checks; special accommodations and faculty-availability constraints add irregular, hard-to-satisfy restrictions that can make regions of the search space infeasible; and real institutions require the algorithm to run quickly and to produce a usable timetable even under time pressure.

For scalability, the recommended strategy is backtracking search enhanced with strong constraint propagation (forward checking or arc-consistency) combined with intelligent variable- and value-ordering heuristics — the Minimum Remaining Values (MRV) heuristic (schedule the most constrained exam first) and the Least Constraining Value heuristic (choose the (slot, hall) that leaves the most flexibility for other exams). For very large universities, this is often further combined with local-search repair (e.g., min-conflicts) to quickly patch a near-complete timetable, or with metaheuristics (simulated annealing/genetic algorithms) when a fully systematic search becomes too slow, giving a good balance between correctness guarantees and practical runtime.

Answer 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax Models This Problem

Minimax models the game as a two-player, zero-sum decision tree where nodes alternate between the AI's (maximizing) turn and the human opponent's (minimizing) turn. At each MAX node, the algorithm assumes the AI will choose the move leading to the highest evaluated outcome; at each MIN node, it assumes the (adversarial) human opponent will choose the move that is worst for the AI, i.e., lowest evaluated outcome. Values are computed by recursing to the terminal states (or a fixed search depth) and propagating evaluation scores back up the tree, so the AI's chosen move at the root is the one that is best assuming the opponent always plays optimally against it — a worst-case, safety-first model of adversarial play.

ii. Computational Challenges of Large Game Trees

The size of a game tree grows exponentially with depth: with branching factor b and search depth d , a full Minimax search must examine on the order of b^d nodes. In a real-time strategy game with many possible unit actions per turn, b can be very large, so even modest search depths become computationally infeasible within the strict time limits the AI must obey. This forces trade-offs between search depth (looking further ahead for better decisions) and response time (needed to act in real time), and makes full-tree exhaustive Minimax impractical without some form of pruning or depth limitation.

iii. Alpha-Beta Pruning

Alpha-Beta pruning augments Minimax with two bounds — alpha (the best value the maximizer can guarantee so far) and beta (the best value the minimizer can guarantee so far) — passed down the recursion. While exploring a MIN node, if a value is found that is less than or equal to alpha, the remaining siblings at that node can be skipped entirely, because the maximizer

already has a better alternative elsewhere and the minimizer would never let play reach a worse-for-it branch; symmetrically, a MAX node value greater than or equal to beta lets the minimizer prune the remaining branches. This never changes the final decision reached compared to full Minimax — it only avoids expanding branches that cannot possibly affect the outcome — and with good move ordering (examining likely-best moves first) it can reduce the effective branching factor from b to roughly $\sqrt[b]{b}$, allowing the same time budget to search substantially deeper than plain Minimax.

iv. Designing an Evaluation Function

Because the full game tree cannot be searched to terminal states in real time, a heuristic evaluation function is used at the cutoff depth to estimate how favourable a game state is for the AI. Reasonable factors to include are: material/unit balance (number and strength of surviving units on each side, weighted by unit value); positional factors (map control, terrain advantage, proximity to strategic resources or objectives); resource and economic strength (gathered resources, production capacity); tactical factors (threats to key units, vulnerability of the AI's base, mobility/tempo advantage); and a game-progress term that shifts weighting over time (e.g., early-game economy vs late-game combat strength). These factors are combined into a weighted sum, and the weights themselves are typically tuned through playtesting, self-play, or learning, so the function reflects which factors most reliably predict eventual victory.

v. Balancing Optimality and Real-Time Performance

Given the strict time limits, the recommended strategy is Alpha-Beta pruning with iterative deepening — searching to depth 1, then 2, then 3, and so on, always keeping the best move found so far, and stopping (returning the best move found) the instant the time budget expires; this guarantees the AI always has a legal, reasonably good move ready regardless of how deep it manages to search. Combining this with good move ordering (trying captures/high-value moves first to maximize pruning), a transposition table (to reuse evaluations of previously seen states), and a well-tuned evaluation function at the cutoff depth gives the best practical trade-off: near-optimal play when time allows, and still-competent play when time is tight, rather than either freezing while searching for a perfect move or acting on a weak, shallow evaluation.